

# RedTail-X: Variable-Length Final Shares for 4+2 Reed–Solomon Erasure-Coded Storage

Michael Cohee

*Independent Researcher, New York, NY, USA*

michaelacohee@gmail.com

## Abstract

A common implementation of 4+2 Reed–Solomon (RS) erasure coding stripes data into four fixed-size data shares and computes two parity shares, zero-filling any stripe that is shorter than a full stripe. For corpora with many small files, the zero-fill in each file’s final stripe is itself erasure-coded and stored, and it dominates storage overhead. RedTail-X is a storage-layout rule that leaves full stripes unchanged and sizes the six shares of the final partial stripe to  $\lceil t/4 \rceil$  bytes, where  $t$  is the number of bytes in that stripe. A per-stripe manifest records the original length so that the decoder can truncate exactly. On a corpus of 213 PDF files (666,933,563 bytes) on a single Apple M1 volume, the coded output fell from  $1.8277\times$  to  $1.5000\times$  of input, a reduction of 218,568,756 bytes (17.93%), while all partial tails were reconstructed byte-for-byte after simulated two-share loss. The RS mathematics are unchanged; the contribution is an empirical layout result. We also report negative results for several candidate formulations that were tested and excluded, and we state the scope limits of a single-volume, single-corpus study.

**Keywords:** erasure coding; Reed–Solomon; storage overhead; padding; small files; distributed storage; Rust

## 1. Introduction

Erasure coding protects stored data against the loss of storage devices or nodes at a lower storage cost than replication. A systematic  $(n, k) = (6, 4)$  Reed–Solomon code [1], written “4+2”, stores four data shares and two parity shares per stripe, tolerates the loss of any two of the six shares, and has a nominal storage rate of  $n/k = 1.5$ .

That nominal rate holds only when every stripe is full. In the DarkRock prototype studied here, each stripe carries up to 1 MiB of input as four 256 KiB data shares. When a file’s last stripe is shorter than 1 MiB, the implementation zero-filled it to a full stripe before encoding, so a 10 KB file emitted six 256 KiB shares (1.5 MiB). On a PDF corpus the measured coded output was  $1.8277\times$  the input instead of  $1.5\times$ . The excess is caused entirely by encoding and storing zero bytes.

**This report makes three contributions:**

- **A layout rule (RedTail-X)** that sizes only the final stripe’s shares to  $\lceil t/4 \rceil$  bytes, with a manifest that makes the variable length unambiguous at decode time (Section 4).
- **Measurements** on a 213-file PDF corpus showing the coded ratio returning to  $1.5000\times$ , together with tail encode and repair timings and a compression-pipeline result (Section 5).
- **Documented negative results** for formulations that were proposed, implemented, measured and excluded (Section 6), and a repair-control defect found during testing (Section 7).

RedTail-X is not a new erasure code. Shortening the final segment or block is established practice in some systems (Section 3). The value of this report is a measured, reproducible account of the effect in one implementation, including what did not work.

## 2. Background

### 2.1 Systematic 4+2 Reed–Solomon

Let  $d \in \mathbb{F}^4$  be four data symbols over the byte field  $\mathbb{F} = GF(2^8)$ . A systematic 4+2 code computes a codeword  $c = G d$  with generator matrix  $G = [I_4 ; C] \in \mathbb{F}^{6 \times 4}$ , where the first four outputs are the data unchanged and the last two are parity.  $G$  is chosen (for example from a Vandermonde or Cauchy construction [4]) so that every  $4 \times 4$  submatrix is invertible. The code is therefore maximum distance separable (MDS) [2]: any four of the six shares recover  $d$ . Applied byte-wise across equal-length shares, the stored size is  $6/4 = 1.5$  times the encoded payload.

The implementation evaluated here uses the Rust crate `reed-solomon-erasure` [15] in a systematic Vandermonde configuration over  $GF(2^8)$ . RedTail-X does not change  $G$ , the field, or the decoding procedure.

### 2.2 Byte entropy

For a byte string  $x$  of length  $N$ , let  $p(\sigma)$  be the empirical frequency of byte value  $\sigma \in \{0, \dots, 255\}$ . The Shannon entropy [3] in bits per byte is

$$H_8(x) = - \sum_{\sigma} p(\sigma) \log_2 p(\sigma), \quad 0 \leq H_8 \leq 8, (1)$$

with the convention  $0 \cdot \log 0 = 0$ . Low values indicate repetitive, compressible data; values near 8 indicate compressed, encrypted or random-looking data. Section 5.4 uses  $H_8$  only as a gate that decides whether to attempt compression; it plays no role in erasure coding.

## 3. Related Work

**Reed–Solomon and MDS codes.** Reed and Solomon introduced polynomial codes over finite fields [1]; the Singleton bound characterizes MDS codes [2]. Plank’s tutorial describes practical RS coding for storage [4].

**Final-segment handling.** Tahoe-LAFS divides a file into segments, and its specification notes that the last segment, and therefore the last block of each share, is likely to be shorter than the rest [6]. RedTail-X applies the same principle to a fixed 256 KiB share design and quantifies its effect on a small-file-heavy corpus.

**Decentralized and cloud storage.** Storj uses Reed–Solomon erasure shares with considerably wider parameters than 4+2 [7]. Locally repairable codes reduce repair reads at the cost of extra parity [8], and regenerating codes minimize repair bandwidth [9]. RedTail-X is orthogonal: it changes only how much zero-fill is encoded.

**Two-parity constructions.** A parity pair  $P = \oplus d_i$ ,  $Q = \oplus g^i d_i$  over  $GF(2^8)$  is the RAID-6 construction described by Anvin [5]. The custom parity evaluated in Section 5.3 belongs to this family.

**Compression heuristics.** The Btrfs file system decides whether to compress using data sampling, repeated-pattern detection, byte frequency and Shannon entropy [10]. The compression gate in Section 5.4 follows the same idea; its specific threshold and sample placement are measured here. Compression uses Zstandard, developed by Yann Collet at Meta (formerly Facebook) [11], [17].

## 4. Design

### 4.1 Stripe layout

*A file of  $L$  bytes (after any optional compression; Section 5.2) is split into  $\lfloor L / 2^{z^0} \rfloor$  full stripes and, if  $t = L \bmod 2^{z^0} > 0$ , one partial tail of  $t$  bytes.*

- **Full stripe:** four 256 KiB data shares and two 256 KiB parity shares (1 MiB input, 1.5 MiB coded). Unchanged from the baseline.
- **Partial tail (baseline):** zero-filled to 1 MiB, then encoded as a full stripe. Coded size is always 1.5 MiB.
- **Partial tail (RedTail-X):** each of the six shares has length

$$s = \lceil t / 4 \rceil, \quad \text{coded bytes} = 6s(2)$$

The four data shares hold the tail in order, so at most 3 bytes of zero-fill occur in total across the four data shares. The coded size of the tail is  $6 \lceil t/4 \rceil \leq 1.5t + 4.5$  bytes.

### 4.2 Manifest and decoding

*The share length alone does not determine the original length: up to four values of  $t$  map to the same  $s$ . Each `StripeManifest` therefore records `original_len`, `share_len`, a hash of the original stripe bytes, and a hash of each of the six shares. The decoder:*

1. checks that `share_len = \lceil original\_len / 4 \rceil`;
2. checks the length and hash of every available share, treating a mismatched share as an erasure;
3. reconstructs from any four valid shares;
4. truncates the result to `original_len`; and
5. verifies the hash of the original bytes.

The hashes detect accidental corruption only. They do not authenticate data against an adversary (Section 8).

## 5. Evaluation

### 5.1 Setup

*All measurements ran on a single Apple M1 computer using Rust release builds (Cargo.lock format version 4) with the crates `reed-solomon-erasure 6.x` [15], `zstd 0.13` [18] (Rust bindings to the Zstandard library developed at Meta [11], [17]) and `sha2 0.10` [19] for SHA-256 [20], on one local volume, CPU only, with no network, disk-persistence or encryption layer in the measured path. Timings are single-pass observations and should be read as indicative. The primary corpus is 213 PDF files totalling 666,933,563 bytes.*

### 5.2 Storage overhead

*Table 1. Emitted share bytes for 213 PDF files (666,933,563 input bytes).*

| Layout                          | Emitted share bytes | Ratio to input |
|---------------------------------|---------------------|----------------|
| Fixed 256 KiB shares (baseline) | 1,218,969,600       | 1.8277×        |
| RedTail-X variable final shares | 1,000,400,844       | 1.5000×        |
| Reduction                       | 218,568,756         | −17.93%        |

*The ideal 1.5× of input is 1,000,400,344.5 bytes; RedTail-X exceeds it by 499.5 bytes in total, from per-file rounding. PDFs smaller than 1 MiB held 7.38% of input bytes but caused 70.57% of the baseline’s padding above 1.5×, confirming that small files drive the overhead.*

*Tail encoding time fell from 1,338.0 ms to 611.5 ms, and tail repair after a two-share loss fell from 1,358.9 ms to 693.2 ms, because less zero-fill is processed. Full-stripe timings were essentially unchanged.*

*Compression pipeline. With the compression gate of Section 5.4 (8 KiB start sample, zstd level 3, keep the smaller verified representation) placed before RedTail-X, the selected representation was 649,644,262 bytes and the RS share bytes were 974,466,894: 1.5000× the selected bytes and 1.4611× the original PDF bytes.*

*Alternative: batching all files. Concatenating the whole corpus into one stream with fixed shares would emit 1,001,914,368 bytes (a saving of 217,055,232), slightly worse than per-file RedTail-X, and would additionally require a pack index and policies for update, deletion and garbage collection. It was not implemented.*

### 5.3 Custom parity versus library RS

*A custom two-parity  $GF(2^8)$  construction (a plain XOR sum and a weighted sum of the four data shares, i.e. the RAID-6  $P+Q$  form [5]) was implemented and compared with the library on 16 MiB of pseudorandom data.*

*Table 2. Standard RS versus custom parity, 16 MiB, single pass.*

| Operation                       | Standard RS | Custom parity |
|---------------------------------|-------------|---------------|
| Encode                          | 140.44 ms   | 193.91 ms     |
| Repair, one data share missing  | 165.50 ms   | 134.39 ms     |
| Repair, two data shares missing | 155.80 ms   | 235.20 ms     |
| Coded storage                   | 1.50×       | 1.50×         |

*The construction is in the systematic MDS family and showed no storage or repair-bandwidth advantage. It is retained only as a test oracle; production encoding uses the library.*

#### 5.4 Compression gate

*The gate examines one 8 KiB sample at the start of each 256 KiB chunk and attempts zstd level 3 if  $H_8 < 6.4$  bits/byte or a same-sample periodicity check finds a strong repeat; the smaller verified representation is kept. Policies were compared on a mixed corpus of 477 files (872,272,600 bytes).*

*Table 3. Compression-gate policies, 477 files.*

| Policy              | Bytes saved | Missed vs. always-zstd | Median select time |
|---------------------|-------------|------------------------|--------------------|
| 8 KiB start sample  | 36.6 MB     | 21.7 MB                | 3.35 s             |
| 8 KiB middle sample | 32.2 MB     | 26.1 MB                | 3.19 s             |
| Always zstd         | 58.3 MB     | 0                      | 6.32 s             |

*The start-sample policy saved the most bytes among the sampled policies at roughly half the selection time of always compressing. The gate is independent of RedTail-X: it changes which bytes enter the encoder, not how shares are sized.*

#### 5.5 Repair-helper routing (simulation)

*In 1,000 synthetic network topologies, choosing repair helpers by latency-weighted shortest path [12] instead of first-available beat first-available in 857 trials, tied in 130 and lost in 13, with a median modelled time ratio of 0.841. This is a model result; no network repair was executed.*

## 6. Excluded Formulations

The following ideas were proposed during the project, implemented or analysed, and excluded. They are reported because negative results constrain future work.

Zero-coefficient polynomial canonicalization (“the knob”). Content was represented symbolically as  $P(x) = \sum c_e x^e$  with zero coefficients omitted; the implementation used  $\epsilon 64$  and discarded  $|c_e| < 10^{-9}$ . It served as a check stage during development. Its canonical form combines like terms, drops zero terms and sorts before hashing, which is valid for polynomials because addition is commutative. Byte streams are not commutative: order-dependent content such as machine instructions (assembly) or zero-byte sequences becomes indistinguishable after sorting, so distinct inputs could be merged. The formulation was excluded because it preserves a value, not a file: exact reassembly of the original bytes would require per-object undo information, and no byte-preserving transform or net storage saving was established.

N\*Tropy (corpus-normalized entropy). We tested the score  $H_n = H_8 / \log_2 N$ , where  $N$  is the number of chunks in the corpus, which is related to normalized entropy or evenness [13]. It mis-scales with corpus size: measured  $H_8$  barely changed (7.8061 to 7.8038 bits/byte) between 1,239 and 2,477 chunks while  $H_n$  fell from 0.7597 to 0.6922. A logit transform  $\ln(p/(1-p))$  considered alongside it is undefined at  $p \in \{0, 1\}$ . The fixed-threshold gate of Section 5.4 was retained instead.

Clamped entropy estimator. Clamping each frequency to  $[\epsilon, 1-\epsilon]$  was proposed to keep logarithms finite. After clamping, the 256 frequencies no longer sum to 1, so the estimate can exceed 8 (a file of one repeated byte with  $\epsilon = 0.01$  scores about 17 bits). The correct forms are the convention  $0 \cdot \log 0 = 0$  in Eq. (1), or renormalization after clamping.

Shortest-form representation with tight entropy bounds. A rule that keeps one control plus the shortest byte-length representation among equivalent forms, followed by a two-sided entropy threshold near 100%, ran faster than the static pipeline but stored more bytes, because most chunks were refused, zstd verification was skipped, and data was stored raw.

## 7. Correctness and Reliability

Unit tests cover lengths from 1 byte to full-stripe boundaries and every one-share and two-share loss pattern (6 + 15 patterns). In the PDF benchmark, all 213 partial tails were reconstructed and compared byte-for-byte after simulated loss of data share 0 and parity share 4. In the combined compression pipeline, every coded stripe was reconstructed after the same loss pattern and every file was restored and compared byte-for-byte.

**Mid-repair failure.** Separate lab testing found that the repair process could report success while an independent full audit showed five files still missing one share. Repair now runs a final-state audit that reports residual missing shares, and a third overlapping loss stops repair with an explicit error (“lost >2 shares”). This is a repair-control defect, not a coding defect, and it shows that at scale, overlapping failures must be handled by repair epochs and audits rather than by assuming isolated two-share losses.

## **8. Limitations and Future Work**

- One local file-size distribution (PDFs) on one volume of one machine; savings depend on the share of small files in a workload.
- Single-pass timings; no disk persistence, network repair, encryption or node placement in the measured path.
- The manifest was held in memory and is not a persistent, authenticated format. Hashes detect accidental corruption only; a later lab variant uses HMAC-SHA256 [14] with the key stored on the same machine.
- Repair tests used one two-loss pattern at corpus scale (unit tests cover all patterns).

The planned next step holds 4+2, rate 1.5 and  $\lceil t/4 \rceil$  fixed and tests scale: a 1 TB run with persisted shares, an authenticated on-disk manifest, process-isolated storage nodes, seeded one- and two-node losses including an overlapping loss during repair, and byte-for-byte restore of every object, reporting manifest bytes per terabyte and placement cardinality. Packing short tails across files is a candidate later experiment.

## 9. Conclusion

Zero-filling the final stripe of each file dominated the storage overhead of a fixed-share 4+2 Reed–Solomon layout on a small-file-heavy PDF corpus. Sizing only the final stripe’s shares to  $\lceil t/4 \rceil$  bytes and recording the original length in a manifest restored the coded ratio from  $1.8277\times$  to  $1.5000\times$  and reduced output by 17.93%, with byte-exact recovery after two-share loss in local tests. This is a storage-layout result, not a new erasure code, and not evidence that the prototype outperforms deployed decentralized storage networks.

### Author Contributions and Use of AI Tools

The author conceived and directed the project, set its scope and questions, commissioned and ran every experiment on his own hardware, chose to retain the 4+2 parameters and the 256 KiB block size (carried over from his earlier storage code), proposed the candidate formulations in Section 6 (the use of polynomial canonicalization as a check stage,  $N^*$ Tropy, the clamped entropy estimator and the shortest-form rule), decided which results to accept or exclude, and verified all reported results.

AI assistants were used throughout: Claude (Anthropic), ChatGPT and its coding agent (OpenAI), Grok (xAI) and DeepSeek. The removal of final-stripe zero-fill (Section 4) was developed jointly by the author and AI assistants. AI tools implemented the Rust code, including the benchmark harness, which the coding agent ran on the author’s machine at his direction. They formalized the mathematics, explained background theory, reviewed the evaluation methodology, and identified the normalization error in the clamped estimator. They also drafted the text of the original blog post and of this report. The zero-coefficient canonicalization code (combine like terms, drop zeros, sort, hash) was proposed by DeepSeek. The author identified that it fails on order-dependent content such as assembly, and it was excluded (Section 6). No AI system is listed as an author. The author takes full responsibility for the content.

### Data and Code Availability

Source code: <https://github.com/michaelcohee/DarkRock>. An earlier write-up of these results appeared on the author’s blog [16]. The PDF corpus is not redistributable; per-file sizes sufficient to reproduce Table 1 can be provided.

## References

[1] I. S. Reed and G. Solomon, “Polynomial codes over certain finite fields,” *J. Soc. Ind. Appl. Math.*, vol. 8, no. 2, pp. 300–304, 1960.



- [2]R. C. Singleton, “Maximum distance q-nary codes,” *IEEE Trans. Inf. Theory*, vol. 10, no. 2, pp. 116–118, 1964.
- [3]C. E. Shannon, “A mathematical theory of communication,” *Bell Syst. Tech. J.*, vol. 27, pp. 379–423, 623–656, 1948.
- [4]J. S. Plank, “A tutorial on Reed–Solomon coding for fault-tolerance in RAID-like systems,” *Softw. Pract. Exper.*, vol. 27, no. 9, pp. 995–1012, 1997.
- [5]H. P. Anvin, “The mathematics of RAID-6,” kernel.org, 2004 (rev. 2011). <https://mirrors.edge.kernel.org/pub/linux/kernel/people/hpa/raid6.pdf>
- [6]Tahoe-LAFS Project, “File Encoding,” Tahoe-LAFS 1.12.1 documentation. <https://tahoe-lafs.readthedocs.io/en/tahoe-lafs-1.12.1/specifications/file-encoding.html>
- [7]Storj Labs, “Storj: A Decentralized Cloud Storage Network Framework,” white paper v3.0, 2018.
- [8]C. Huang, H. Simitci, Y. Xu, A. Ogus, B. Calder, P. Gopalan, J. Li and S. Yekhanin, “Erasure coding in Windows Azure Storage,” in *Proc. USENIX ATC*, 2012.
- [9]A. G. Dimakis, P. B. Godfrey, Y. Wu, M. J. Wainwright and K. Ramchandran, “Network coding for distributed storage systems,” *IEEE Trans. Inf. Theory*, vol. 56, no. 9, pp. 4539–4551, 2010.
- [10]Btrfs Project, “Compression,” BTRFS documentation. <https://btrfs.readthedocs.io/en/latest/Compression.html>
- [11]Y. Collet and M. Kucherawy, “Zstandard Compression and the ‘application/zstd’ Media Type,” RFC 8878, IETF, 2021.
- [12]E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numer. Math.*, vol. 1, pp. 269–271, 1959.
- [13]E. C. Pielou, “The measurement of diversity in different types of biological collections,” *J. Theor. Biol.*, vol. 13, pp. 131–144, 1966.
- [14]H. Krawczyk, M. Bellare and R. Canetti, “HMAC: Keyed-Hashing for Message Authentication,” RFC 2104, IETF, 1997.
- [15]D. Ldl and contributors, “reed-solomon-erasure: Rust implementation of Reed–Solomon erasure coding,” version 6, MIT License, rust-rse project, GitHub. <https://github.com/rust-rse/reed-solomon-erasure> (a port of Backblaze’s Java, Klaus Post’s Go and Nicolas Trangez’s Haskell implementations).
- [16]M. Cohee, “RedTail-X: Variable-Length Final Shares for 4+2 Erasure-Coded Storage,” *Premiseworks* (blog), Sept. 27, 2026. <https://premiseworks.blogspot.com/2026/09/redtail-x-variable-length-final-shares.html>
- [17]Meta Platforms, Inc., “Zstandard – Fast real-time compression algorithm,” GitHub repository. <https://github.com/facebook/zstd>
- [18]“zstd” Rust crate, version 0.13 (bindings to the Zstandard C library), docs.rs. <https://docs.rs/zstd>
- [19]RustCrypto Project, “sha2” Rust crate, version 0.10, docs.rs. <https://docs.rs/sha2>

**[20]National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, 2015.**